

Assignment # 2

남현욱

2021-25033

1. Solve and attach your solving procedures on the following problems on the textbook:

A. 4.1 (a), (b), (c), & (d)- Engineering economics

4.1(a)

Present Value = (Return) / (1 + interest rate) ^ period

* Project A

$$V0(5) = (1,000,000) / (1 + 0.1) ^ (5) = 620,921$$

$$V0(10) = (4,000,000) / (1 + 0.1) ^ (10) = 1,542,173$$

$$V0(\text{total}) = 2,163,094$$

* Project B

$$V0(5) = (2,000,000) / (1 + 0.1) ^ (5) = 1,241,843$$

$$V0(10) = (3,000,000) / (1 + 0.1) ^ (10) = 1,156,630$$

$$V0(15) = (2,000,000) / (1 + 0.1) ^ (15) = 487,784$$

$$V0(\text{total}) = 2,287,257$$

4.1(b)

* Project A

Total present value of net benefit V0P =

Present value - initial costs = -369,906

$$\begin{aligned} \text{Annual net benefit} &= -369,906 * [0.1 * (1+0.1)^{10}] / [(1+0.1)^{10} - 1] \\ &= -60,200 \end{aligned}$$

* Project B

Total present value of net benefit V0P =

Present value - initial costs = -1,122,743

$$\text{Annual net benefit} = -1,122,743 * [0.1 * (1+0.1)^{15}] / [(1+0.1)^{15} - 1]$$

$$= -147,611$$

4.1(c)

이자율이 0.1%인 경우 Project A의 Present Value 와 Annual net benefit의 값이 더 크며

이자율이 0.05%인 경우에도

Project A 의 Present Value는 706,179\$ 이며 Annual net benefit 은 91,453\$이고 Project B 의 Present Value는 370,826\$ 이며 Annual net benefit 은 35,726\$로 Project A의 값이 더 크다.

따라서 이자율이 0.05%로 조정되어도 Project A가 더 선호된다.

4.1(d)

* Project A

$$V0(5) = (1,000,000) / (1 + 0.1)^{(5)} = 620,921$$

$$V0(10) = (4,000,000) / (1 + 0.1)^{(10)} = 1,542,173$$

$$V0(15) = (1,000,000) / (1 + 0.1)^{(15)} = 239,392$$

$$V0(20) = (4,000,000) / (1 + 0.1)^{(20)} = 594,575$$

$$V0(25) = (1,000,000) / (1 + 0.1)^{(25)} = 92,296$$

$$V0(30) = (4,000,000) / (1 + 0.1)^{(30)} = 229,234$$

$$V0(\text{total}) = 3,318,591$$

Total present value of net benefit V0P =

$$\text{Present value} - \text{initial costs} = 3,318,591 - (2,533,000 + 2,533,000/(1+0.1)^{10} + 2,533,000/(1+0.1)^{20}) = -567,504$$

$$\begin{aligned} \text{Annual net benefit} &= -567,504 * [0.1 * (1+0.1)^{30}] / [(1+0.1)^{30} - 1] \\ &= -60,200 \end{aligned}$$

* Project B

$$V0(5) = (2,000,000) / (1 + 0.1)^{(5)} = 1,241,843$$

$$V0(10) = (3,000,000) / (1 + 0.1)^{(10)} = 1,156,630$$

$$V0(15) = (2,000,000) / (1 + 0.1)^{(15)} = 487,784$$

$$V0(20) = (2,000,000) / (1 + 0.1)^{(20)} = 297,287$$

$$V0(25) = (3,000,000) / (1 + 0.1)^{(25)} = 276,888$$

$$V0(30) = (2,000,000) / (1 + 0.1)^{(30)} = 114,617$$

$$V0(\text{total}) = 3,566,049$$

Total present value of net benefit V0P =

$$\begin{aligned} \text{Present value} - \text{initial costs} &= 3,566,048 - (4,000,000 - 4,000,000/(1+0.1)^{15}) \\ &= -1,391,519 \end{aligned}$$

$$\begin{aligned} \text{Annual net benefit} &= -1,391,519 * [0.1 * (1+0.1)^{30}] / [(1+0.1)^{30} - 1] \\ &= -147,611 \end{aligned}$$

그러므로 (b)에서 구한 Annual net benefit 과 값이 같다.

B. 4.9 (a) & (b)- Lagrange multiplier

u.a. 3 users.

$$\begin{aligned} P(x_1) &= 12x_1 - x_1^2 \\ P(x_2) &= 8x_2 - x_2^2 \\ P(x_3) &= 16x_3 - 3x_3^2 \end{aligned}$$

ob $\rightarrow$ maximize. $f(x)$
 cons $\rightarrow x_1 + x_2 + x_3 \leq 10$.

$$L = 12x_1 - x_1^2 + 8x_2 - x_2^2 + 16x_3 - 3x_3^2 - \lambda (x_1 + x_2 + x_3 - 10 + s^+)$$

$$\frac{\partial L}{\partial x_1} = 12 - 2x_1 - \lambda = 0 \quad \dots \textcircled{1}$$

$$\frac{\partial L}{\partial x_2} = 8 - 2x_2 - \lambda = 0 \quad \dots \textcircled{2}$$

$$\frac{\partial L}{\partial x_3} = 16 - 6x_3 - \lambda = 0 \quad \dots \textcircled{3}$$

$$\frac{\partial L}{\partial \lambda} = -(x_1 + x_2 + x_3 - 10 + s^+) = 0 \quad \dots \textcircled{4}$$

$$\textcircled{1} \rightarrow 12 - 2x_1 = \lambda$$

$$\textcircled{2} \rightarrow 8 - 2x_2 = \lambda$$

$$\textcircled{2} \rightarrow 8 - 2x_2 = \lambda$$

$$\textcircled{3} \rightarrow 16 - 6x_3 = \lambda$$

$$\rightarrow 12 - 2x_1 = 8 - 2x_2$$

$$\rightarrow 8 - 2x_2 = 16 - 6x_3$$

$$\rightarrow -2x_1 = -4 - 2x_2$$

$$\rightarrow -6x_3 = -8 - 2x_2$$

$$\rightarrow x_1 = 2 + x_2$$

$$\rightarrow x_3 = \frac{4}{3} + \frac{1}{3}x_2$$

$$\Rightarrow -(2 + x_2 + \frac{4}{3} + \frac{1}{3}x_2 - 10) = 0$$

$$\rightarrow 2x_2 + \frac{10}{3} + \frac{1}{3}x_2 - 10 = 0$$

$$\rightarrow \frac{11}{3}x_2 = \frac{20}{3}$$

$$\rightarrow x_2 = \frac{20}{n}$$

$$x_1 = 2 + \frac{20}{n} = \frac{34}{n}$$

$$x_3 = \frac{4}{3} + \frac{1}{3} \times \frac{20}{n} = \frac{20}{21} + \frac{20}{21} = \frac{40}{21} = \frac{16}{n}$$

$$x_1 + x_2 + x_3 = \frac{70}{n} = \underline{10}$$

$$(u) \quad x_1 = \frac{34}{n}, \quad x_2 = \frac{20}{n}, \quad x_3 = \frac{16}{n}$$

$$\begin{aligned} (b) \quad \textcircled{1} \rightarrow 12 - 2x_1 - \lambda &= 0 \\ \rightarrow 12 - 2 \times \frac{34}{n} &= \lambda \\ \rightarrow 12 - \frac{68}{n} &= \lambda \\ \rightarrow \lambda &= \frac{16}{n} \end{aligned}$$

$$\begin{aligned} \textcircled{2} \rightarrow 8 - 2x_2 - \lambda &= 0 \\ \rightarrow 8 - 2 \cdot \frac{20}{n} &= \lambda \\ \rightarrow 8 - \frac{40}{n} &= \lambda \\ \rightarrow \lambda &= \frac{16}{n} \end{aligned}$$

$$\begin{aligned} \textcircled{3} \rightarrow 16 - 6x_3 - \lambda &= 0 \\ \rightarrow 16 - 6 \cdot \frac{16}{n} &= \lambda \\ \rightarrow 16 - \frac{96}{n} &= \lambda \\ \rightarrow \lambda &= \frac{16}{n} \end{aligned}$$

(C)

$$R'(x_1) = 12 - 2x_1$$

$$R'(x_2) = 8 - 2x_2$$

$$R'(x_3) = 18 - 6x_3$$

$\therefore x_1 = 6, x_2 = 4, x_3 = 3$ 일때

$R(x_1) + R(x_2) + R(x_3)$ 는 최댓값 가정함.

$$\Rightarrow (12 \times 6 - 36) + (8 \times 4 - 16) + (18 \times 3 - 3 \times 9)$$

$$= \boxed{79}$$

C. 4.15- Dynamic programming (deterministic)

R coding

```
T.S=array(20,c(4,4))
```

```
T.R=array(25,c(4,4))
```

```
ini.s=c(0,10,20,30)
```

```
St=array(ini.s, c(4,4))
```

```
St_A=array(c(0,0,0,0,10,10,10,10,20,20,20,20,30,30,30,30), c(4,4))
```

```
Q1 = array(c(rep(10,16)), c(4,4))
```

```
Q2 = array(c(rep(50,16)), c(4,4))
```

```
Q3 = array(c(rep(20,16)), c(4,4))
```

```
R1 = Q1 + St - St_A
```

```
R2 = Q2 + St - St_A
```

```
R3 = Q3 + St - St_A
```

```
R1[R1<10 | R1>40] = NA
```

```
R2[R2<10 | R2>40] = NA
```

```
R3[R3<10 | R3>40] = NA
```

```
#TSD
```

```
TSD1 = array(0,c(4,4))
```

```
for(i in 1:4){
```

```
  for(j in 1:4){
```

```
    TSD1[i,j] = (St[i,j]-T.S[i,j])^2 + (St_A[i,j]-T.S[i,j])^2 + (R1[i,j]-T.R[i,j])^2
```

```
  }
```

```
}
```

```
TSD2 = array(0,c(4,4))
```

```
for(i in 1:4){
```

```
  for(j in 1:4){
```

```
    TSD2[i,j] = (St[i,j]-T.S[i,j])^2 + (St_A[i,j]-T.S[i,j])^2 + (R2[i,j]-T.R[i,j])^2
```

```
  }
```

```
}
```

```
TSD3 = array(0,c(4,4))
```

```
for(i in 1:4){
```

```
  for(j in 1:4){
```

```
    TSD3[i,j] = (St[i,j]-T.S[i,j])^2 + (St_A[i,j]-T.S[i,j])^2 + (R3[i,j]-T.R[i,j])^2
```

```
}  
}
```

```
#Recursion_backward  
TSD.total = list(TSD3, TSD2, TSD1)  
R.total = list(R3, R2, R1)  
f_t = list()  
s_t_A = list()  
r_t = list()  
fsr_result = list(c(),c(),c())
```

```
for(i in 1:4){  
  ft = min(TSD3[i,], na.rm=TRUE)  
  s_find = which(TSD3[i,]==ft)  
  ss_t_A = ini.s[s_find]  
  rr_t = R3[i,s_find]  
  f_t[[i]] = ft  
  s_t_A[[i]] = ss_t_A  
  r_t[[i]] = rr_t  
}
```

```
result1 = data.frame(cbind(f_t), cbind(s_t_A), cbind(r_t))  
a = array(rep(result1[,1], each=4), c(4,4))  
fsr_result[[1]][[1]] = result1
```

```
### calculation
```

```
for(i in 2:40){  
  
  j = i%%3  
  jj = (i-1)%%3  
  
  if(j==0){j=j+3}  
  if(jj==0){jj=jj+3}
```

```
f_model = TSD.total[[j]]  
f_r = R.total[[j]]
```

```
ll=length(fsr_result[[j]])
```

```

ll=length(fsr_result[[j]])
before_re=fsr_result[[j]][[ll]]

re2 = as.numeric(TSD.total[[j]]) + as.numeric(array(rep(before_re[,1],each=4),
dim=c(4,4)))
b = array(re2, dim=c(4,4))

for(k in 1:4){
  ft2 = min(b[k,], na.rm=TRUE)
  if(ft2==Inf){ft2=0}
  s_find2 = which(b[k,]==ft2)
  ss_t_A2 = ini.s[s_find2]
  rr_t2 = R.total[[2]][k,s_find2]
  f_t[[k]]=ft2
  s_t_A[[k]]=c(ss_t_A2)
  r_t[[k]]=c(rr_t2)
}

fts2 = data.frame(cbind(f_t), cbind(s_t_A), cbind(r_t))

fsr_result[[j]][[ll+1]] = fts2

if(ll>1){
  d = unlist(fsr_result[[j]][[ll]][,1]) - unlist(fsr_result[[j]][[ll+1]][,1])
  if (d[1]==d[2] & d[2]==d[3] & d[3]==d[4]) break
}
}

fsr_result[[j]][[ll+1]]

```

(a) find the operating policy

	release season			final storage volume season		
Initial storage	1	2	3	1	2	3
0	40	30, 20	30	10	20, 30	20
10	x	30	40	10	30	20
20	40	40	40	30	30	30
30	x	x	x	30	x	30

(b) Show how the model's recursive equations change depending on whether the decisions are the release or the final storage volumes.

Decision variable이 release일 경우 하나의 simulation안에 모든 initial storage를 지정할 수 있기 때문에 한번의 calculation으로 모든 경우의 수가 계산 가능하나,

Decision variable이 final storage일 경우 최초의 initial storage별로 simulation이 나와야 한다. 따라서 총 4개 (0, 10, 20, 30)의 simulation이 도출되어야 한다.

(c) Which model do you think is easier to solve? How would each model change if more importance were given to the desired releases than to the desired storage volumes?

Decision variable이 release일 경우 하나의 simulation안에 모든 initial storage를 지정할 수 있기 때문에 한번의 calculation으로 모든 경우의 수가 계산 가능하기 때문에 Decision variable이 release일 경우가 풀기에 더 쉽다.

D. 4.18- Dynamic programming

R=0.1로 가정할 경우 현재가치환산 표는 아래와 같다. 이윤계산기간은 각 기간의 첫 년도로 계산하였다.

Present Value

Units of additional capacity Period years		2	4	6	8	10	Total required capacity at end of period
1	1-5	11	14	16	21	24	2
2	6-10	5	6	7	8		6
3	11-15	2	3				8
4	16-20	1					10

해당 표를 통해 경우의 수를 계산해보았으며, required capacity를 만족시키며 만들 수 있는 최소현재가치는 아래와 같다.

①

년도	1-5	6-10	11-15	16-20
추가용량	6	0	2	2
현재가치	16	0	2	1
총 현재가치	19			

②

년도	1-5	6-10	11-15	16-20
추가용량	2	6	0	2
현재가치	11	7	0	1
총 현재가치	19			

두 경우중 하나의 경우를 고르자면, 소숫점 첫째자리 까지 계산할 경우 ②번의 현재가치가 조금 더 낮기는 하지만 단순 discounted cost를 합산해 보면 ①번의 값이 더 낮기 때문에 ①이 더 좋은 경우라고 볼 수 있다.

①

년도	1-5	6-10	11-15	16-20
추가용량	6	0	2	2
현재가치	18	0	6	4
총 Discounted cost	<u>28</u>			

②

년도	1-5	6-10	11-15	16-20
추가용량	2	6	0	2
현재가치	12	13	0	4
총 Discounted cost	<u>29</u>			

기간에 따라 변화하는 Cost를 구하는 수식은 아래와 같다.

$$C(n+p) = C(n) / (1+r)^p \quad *p = \text{period}$$

2. Solve the following LP Problems in Excel and in R :

A. Attach your R code

```
#install.packages('lpSolve')
library(lpSolve)

# Set coefficients of the objective function
f.obj <- c(10,8,9)

#Set matrix corresponding to coefficients of constraints by rows
#Do not consider the non-negative constraint; it is automatically assumed

f.con <- matrix(c(2,3,1,
                  5,6,6,
                  1,1,1,
                  0,1,0), nrow=4, byrow = TRUE)

#set unequality signs
f.dir <- c("<=",
          "<=",
          "<=",
          ">=")

#set right hand side coefficients
f.rhs <- c(1000,
          2400,
          700,
          160)

#Final value (z)
lp("max", f.obj, f.con, f.dir, f.rhs)

#Variables final value
lp("max", f.obj, f.con, f.dir, f.rhs)$solution
```

B. Compare your results obtained from Excel and R. Are they identical?

Maximize	$10x_1 + 8x_2 + 9x_3$								
Subject to	$2x_1 + 3x_2 + x_3 \leq 1000$		10	8	9				
	$5x_1 + 6x_2 + 6x_3 \leq 2400$								
	$x_1 + x_2 + x_3 \leq 700$		2	3	1	1000	<=	1000	
	$x_1 \geq 0, x_2 \geq 160, x_3 \geq 0$		5	6	6	2400	<=	2400	
			1	1	1	440	<=	700	
			0	1	0	160	>=	160	
			X/Y	240	160	40	4040		

objective function = 4040

$x_1 = 240, x_2 = 160, x_3 = 40$

으로 엑셀과 R의 값이 동일하다.

3. Solve the following DP problem in R:

A. First, set the benefit function of your problem. How would you set your benefit function?

Net Benefit = Total return - Total cost

Release	Benefit(\$)	Total Benefit (\$)	Cost(\$)	Net Benefit(\$)
1	2000	2000	-	2000
2	1500	3500	-	3500
3	1000	4500	-	4500
4	500	5000	-	5000
5	-	5000	1500	3500
6	-	5000	4000	1000

B. What is the governing equation of your problem?

$$S_{t+1} = S_t - r_t + I_t$$

Where,

S_{t+1} = Final storage

S_t = Initial storage

r_t = Release

I_t = Inflow

C. What is the DP backward recursive equation for this example? Your equations should be different for seasons 1-3, and season 4.

$F4(S4) = \text{maximum}(NB4(r4, S4)), \text{ over all } r4 \leq S4 + I4, 0 \leq S4 \leq 4 \text{ (discrete)}$
 $F3(S3) = \text{maximum}(NB3(r3, S3) + F4(S4)), \text{ over all } r3 \leq S3 + I3, 0 \leq S3 \leq 4$
 (discrete), $S4 = S3 - r3 + I3$
 $F2(S2) = \text{maximum}(NB2(r2, S2) + F3(S3)), \text{ over all } r2 \leq S2 + I2, 0 \leq S2 \leq 4$
 (discrete), $S3 = S2 - r2 + I2$
 $F1(S1) = \text{maximum}(NB1(r1, S1) + F2(S2)), \text{ over all } r1 \leq S1 + I1, 0 \leq S1 \leq 4$
 (discrete), $S3 = S1 - r1 + I1$

D. Solve your problem in R. You may solve your problem using Excel First.

#Conditions

ini.s=c(0,1,2,3,4)

St=array(ini.s, c(5,5))

St_A=array(c(0,0,0,0,0,1,1,1,1,1,2,2,2,2,2,3,3,3,3,3,4,4,4,4,4), c(5,5))

I1 = array(c(1,1,1,1,1), c(5,5))

I2 = array(c(2,2,2,2,2), c(5,5))

I3 = array(c(1,1,1,1,1), c(5,5))

I4 = array(c(2,2,2,2,2), c(5,5))

R1=array(0,c(5,5))

R2=array(0,c(5,5))

R3=array(0,c(5,5))

R4=array(0,c(5,5))

for(i in 1:5){

 for(j in 1:5){

 R1[i,j] = if(I1[i,j]+St[i,j]-St_A[i,j] < 0){NA} else{I1[i,j]+St[i,j]-St_A[i,j]}

 R2[i,j] = if(I2[i,j]+St[i,j]-St_A[i,j] < 0){NA} else{I2[i,j]+St[i,j]-St_A[i,j]}

 R3[i,j] = if(I3[i,j]+St[i,j]-St_A[i,j] < 0){NA} else{I3[i,j]+St[i,j]-St_A[i,j]}

 R4[i,j] = if(I4[i,j]+St[i,j]-St_A[i,j] < 0){NA} else{I4[i,j]+St[i,j]-St_A[i,j]}

 }

}

#Release

R=list(R1, R2, R3, R4)

#Net benefit function

```
nbf = c(NA,2000,3500,4500,5000,3500,1000)
ft = list()
```

```
for (i in 1:4){
  #i=1
  R.dat = R[[i]]
  ft.dat = array(NA, c(5,5))

  for (j in 1:5){
    #j=1

    for (k in 1:5){
      #k=1

      r = R.dat[j,k]

      nnbf = nbf[(r+1)]

      ft.dat[j,k] = nnbf

    }

  }

  ft[[i]] = ft.dat
}
```

```
#Recursive_Backward calculation
back.ft =list(ft[[4]], ft[[3]], ft[[2]], ft[[1]])
back.release = list(R4, R3, R2, R1)
ft.result = list(c(),c(),c(),c())
```

```
#First step (T=4)
ft.model = back.ft[[1]]
back.r = back.release[[1]]
ll = length(ft.result[[1]])
ft.l = list()
release.l = list()
storage.l = list()
```

```

for(k in 1:5){
  #k=4
  if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) &
      is.na(ft.model[k,4])){
    next
  }
  s0 = max(ft.model[k,],na.rm=T)
  s0.loc = which(ft.model[k,]==s0)
  ss = s[s0.loc]
  r = back.r[k,s0.loc]

  ft.l[[k]] = s0
  release.l[[k]] = c(r)
  storage.l[[k]] = c(ss)
}

```

```

ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))

```

```

ft.result[[1]][[1]] = ft.r

```

```

#Backward recursive algorithm

```

```

for (i in 2:100){

```

```

  j = mod(i,4)

```

```

  bb = mod((i-1),4)

```

```

  if (j == 0){

```

```

    j = j+4
  }

```

```


```

```

  if (bb == 0){

```

```

    bb = bb+4
  }

```

```


```

```

  ft.model = back.ft[[j]]

```

```

  back.r = back.release[[j]]

```

```

  ll = length(ft.result[[j]])

```

```

  lll = length(ft.result[[bb]])

```

```

  back.step.result = ft.result[[bb]][[lll]]

```

```

ft.l = list()
release.l = list()
storage.l = list()
for (k in 1:5){

  if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) &
      is.na(ft.model[k,4])){
    ft.l[[k]] = NA
    release.l[[k]] = NA
    storage.l[[k]] = NA
    next
  }

  for (l in 1:5){

    if (is.na(ft.model[1,l]) & is.na(ft.model[2,l]) & is.na(ft.model[3,l]) &
        is.na(ft.model[4,l])){
      next
    }
    if (is.na(ft.model[k,l])){
      next
    }
    if (is.na(back.step.result[,1][[l]])){
      ft.model[k,l] = NA
      next
    }
    ft.model[k,l] = ft.model[k,l] + back.step.result[,1][[l]]

  }

  s0 = max(ft.model[k,],na.rm=T)
  s0.loc = which(ft.model[k,]==s0)
  ss = s[s0.loc]
  r = back.r[k,s0.loc]

  ft.l[[k]] = s0
  release.l[[k]] = c(r)
  storage.l[[k]] = c(ss)

}

```

```
ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))

ft.result[[j]][[(ll+1)]] = ft.r

if (ll>2){
  d = unlist(ft.result[[j]][[ll]][,1]) - unlist(ft.result[[j]][[(ll+1)]][,1])
  if (d[1]==d[2] & d[2]==d[3] & d[3]==d[4] & d[4]==d[5]) break
}
}
```

4. Solve the following DP problem in R:

A, B

```
## start ##
#install.packages('xlsx')
#library('xlsx')

data=read.csv('4-Aforr.csv', header=T)

df=data

df

plot(df$관심, type='o', pch=16, col='steelblue', xlab='순', ylab = '관심')

#install.packages("dplyr")
library("dplyr")
#install.packages("zoo")
library("zoo")

#A_Interpolation
interpolation = df %>%
  mutate(관심 = na.approx(관심), 주의 = na.approx(주의), 경계 = na.approx(경계), 심각
    = na.approx(심각))

interpolation

plot(interpolation$관심, type='o', pch=1, col='steelblue', xlab='순', ylab = '관심')

interpolation_sorting=read.csv('0426_interpolation_sorting.csv', header=T)
storage_xls=read.csv('0426_storage.csv', header=T)
inflow = read.csv('0426_inflow_r.csv', header=T)
release = read.csv('0426_release.csv', header=T)
```

```

for(i in 1:365){
  storage_xls[i+1,3] = storage_xls[i,3] + inflow[i,4] -
    if(storage_xls[i,3]>interpolation_sorting[i,3]){release[i,7]
  } else if(storage_xls[i,3]>interpolation_sorting[i,4]){release[i,3]
  } else if(storage_xls[i,3]>interpolation_sorting[i,5]){release[i,4]
  } else if(storage_xls[i,3]>interpolation_sorting[i,6]){release[i,5]
  } else if(storage_xls[i,3]<interpolation_sorting[i,6]){release[i,6]
  }}

```

C.

* left hand side

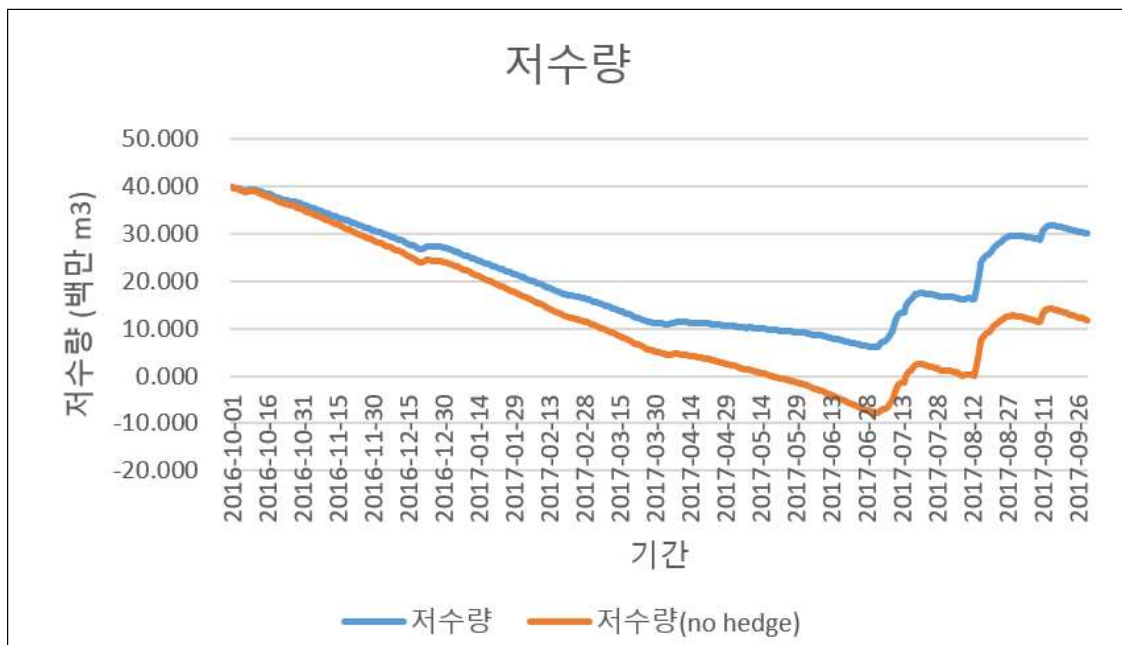
Total inflow + Starting Storage = 120.83 백만 m3

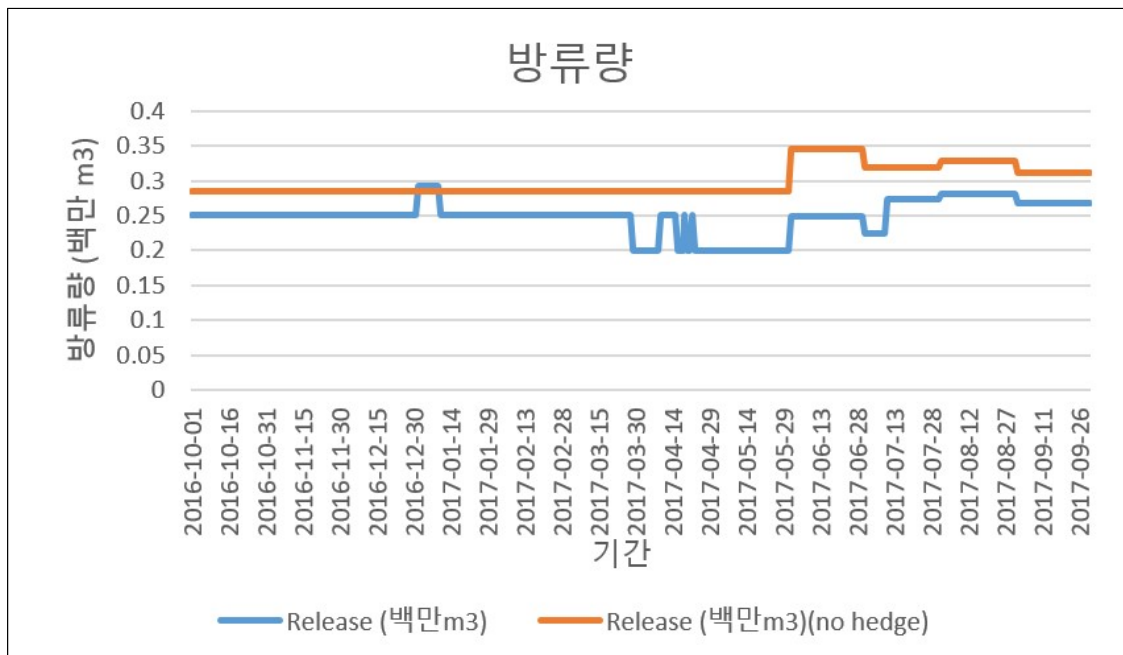
* right hand side

Total Actual Outflow + Final Storage = 120.83 백만 m3

두 값이 같음을 확인할 수 있다.

D.





Quantitatively

- * 헷지룰로 인하여 방류량이 조절되며 이에 따라 저수량이 안정적으로 확보된다.
최소 저수량인 6.2까지 떨어지기 전에 방류량이 조절되기 때문에 정해놓은 기준에 맞추어 안정적이게 운영된다고 볼 수 있다.

Qualitatively

- * 가뭄으로 인해 사용이 불가하게 될 수도 있었던 저수지가 헷지룰로 인해 용수공급이 가능하게 되었으며, 정상 저수량까지 회복속도도 더 빠를 것으로 판단된다.
- * 헷지룰이 없는 경우 3월 25일 부터 최소 저수량을 맞추지 못하며, 5월 19일부터는 용수공급이 불가해지는 반면 헷지룰 적용시 모의운영 기간동안 최소저수량을 만족시키며 용수공급이 가능했다. 이는 댐 운영자의 입장에서 볼때 만족스러운 결과이며, 소비자의 입장에서 사용가능 용수량이 줄게되는 당시에는 불만족스러울수 있으나 향후 용수공급이 불가한 경우를 면 할수 있으므로 장기적으로 볼때에는 소비자에게도 이익이 된다고 볼 수 있다.